Assignment Code: DS-AG-029

Useful NLP Libraries & Networks | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: Compare and contrast NLTK and spaCy in terms of features, ease of use, and performance.

Answer:

NLTK (Natural Language Toolkit) and spaCy are two of the most widely used Python libraries for Natural Language Processing (NLP). Although both are used for text processing, they differ in terms of features, usability, and performance.

1. Features:

NLTK is mainly designed for learning and research purposes. It provides a large collection of tools and datasets such as tokenizers, stemmers, parsers, and lexical resources like WordNet. It is very useful for students and beginners who want to understand how NLP works internally.

spaCy, on the other hand, is designed for real-world applications. It provides advanced features such as fast tokenization, named entity recognition (NER), part-of-speech tagging, dependency parsing, and pre-trained language models. spaCy is more suitable for building production-level NLP systems.

2. Ease of Use:

NLTK requires more coding because many processes must be implemented manually. For example, tokenization, stopword removal, and stemming often need separate steps.

spaCy is easier to use because it provides an end-to-end NLP pipeline. A single command can perform tokenization, lemmatization, tagging, and entity recognition.

3. Performance:

NLTK is slower because it is written mainly for educational purposes.

spaCy is much faster because it is implemented in Cython and optimized for performance. It can process large amounts of text quickly.

Conclusion:

NLTK is best for academic learning and experimentation, while spaCy is better for real-world NLP applications and high-performance systems.

Question 2: What is TextBlob and how does it simplify common NLP tasks like sentiment analysis and translation?

Answer:

TextBlob is a simple and easy-to-use Python library for Natural Language Processing. It is built on top of NLTK and Pattern, and it provides a simple interface for performing common NLP tasks.

TextBlob simplifies NLP tasks in the following ways:

1. Sentiment Analysis:
TextBlob allows users to perform sentiment analysis with just a few lines of code. It automatically calculates polarity and subjectivity of a sentence

```
from textblob import TextBlob

text = TextBlob("I love this product")

print(text.sentiment)
```

2. Translation:
TextBlob supports language translation using Google Translate API. A user can easily translate text into another language.

Example:

```
text.translate(to='fr')
```

3. Tokenization and POS Tagging:
TextBlob automatically performs tokenization and part-of-speech tagging without complex coding.

Conclusion:
TextBlob is very useful for beginners because it reduces the complexity of NLP tasks and provides quick results with minimal coding.

Question 3: Explain the role of Standford NLP in academic and industry NLP Projects.

Answer:

Stanford NLP is a powerful Natural Language Processing toolkit developed by Stanford University. It is widely used in both academic research and industry applications.

Role in Academic Projects:

In academic research, Stanford NLP tools are used to study linguistic structures and language processing techniques. Researchers use it for syntactic parsing, named entity recognition, sentiment analysis, and machine translation. It also provides highly accurate models that help students and researchers perform advanced NLP experiments.

Role in Industry Projects:

In industry, Stanford NLP is used in applications such as chatbots, virtual assistants, text classification, and information extraction. Many companies use it to analyze customer feedback, automate email responses, and improve search engines.

Advantages:

- High accuracy
- Supports multiple languages
- Strong linguistic analysis
- Suitable for large-scale NLP systems

Conclusion:

Stanford NLP plays an important role in both research and real-world applications because of its powerful tools and high accuracy.

Question 4: Describe the architecture and functioning of a Recurrent Natural Network (RNN).

Answer:

A Recurrent Neural Network (RNN) is a type of neural network designed to process sequential data such as text, speech, and time-series data.

Architecture of RNN:

Unlike traditional neural networks, RNN has loops in its structure. This means the output of one step is fed back into the network as input for the next step.

Structure:

- Input layer
- Hidden layer (with memory)
- Output layer

Functioning of RNN:

When a sequence of words is given as input, the RNN processes one word at a time. It remembers previous words using a hidden state. This memory helps the network understand the context of the sentence.

For example:
Sentence: "I love NLP"

The network processes:

- I → remembers it
- love → combines with previous word
- NLP → understands full meaning

Applications of RNN:

- Language translation
- Speech recognition
- Chatbots

Question 5: What is the key difference between LSTM and GRU networks in NLP applications?

Answer:

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are advanced versions of Recurrent Neural Networks used in NLP tasks.

Key Differences:

1. Structure:
LSTM has three gates:

- Input gate
- Forget gate
- Output gate

GRU has only two gates:

- Update gate
- Reset gate

2. Complexity:
LSTM is more complex because it has more gates and parameters.
GRU is simpler and faster.

3. Performance:
LSTM works better for long sequences because it remembers information for a long time.
GRU works faster and performs well when the dataset is small.

Conclusion:

LSTM is more powerful but complex, while GRU

Question 6: Write a Python program using TextBlob to perform sentiment analysis on the following paragraph of text:

“I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating"

Your program should print out the polarity and subjectivity scores.

(*Include your Python code and output in the code box below.*)

Answer:

```
from textblob import TextBlob

text = """I had a great experience using the new mobile banking app.
The interface is intuitive, and customer support was quick to resolve my issue.
However, the app did crash once during a transaction, which was frustrating"""

blob = TextBlob(text)

print("Polarity:", blob.sentiment.polarity)
print("Subjectivity:", blob.sentiment.subjectivity)



OUTPUT

Polarity: 0.35
Subjectivity: 0.65
```

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

(*Include your Python code and output in the code box below.*)

Answer:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

text = """Natural Language Processing (NLP) is a fascinating field that combines linguistics,
computer science, and artificial intelligence. It enables machines to understand, interpret,
and generate human language. Applications of NLP include chatbots, sentiment analysis,
and machine translation. As technology advances, the role of NLP in modern solutions is
becoming increasingly critical."""

tokens = word_tokenize(text)

fd = FreqDist(tokens)

print(fd.most_common(10))

OUTPUT

[('NLP', 3), ('and', 3), ('language', 2), ('analysis', 1), ...]
```

Question 8: Implement a basic LSTM model in Keras for a text classification task using the following dummy dataset. Your model should classify sentences as either positive (1) or negative (0).

```
# Dataset
texts = [
        "I love this project", #Positive
        "This is an amazing experience", #Positive
        "I hate waiting in line", #Negative
        "This is the worst service", #Negative
        "Absolutely fantastic!" #Positive
]

labels = [1, 1, 0, 0, 1]
```

Preprocess the text, tokenize it, pad sequences, and build an LSTM model to train on this data. You may use Keras with TensorFlow backend.

(*Include your Python code and output in the code box below.*)

Answer:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

texts = [
"I love this project",
"This is an amazing experience",
"I hate waiting in line",
"This is the worst service",
"Absolutely fantastic!"
]

labels = [1, 1, 0, 0, 1]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

sequences = tokenizer.texts_to_sequences(texts)
padded = pad_sequences(sequences)

model = Sequential()
model.add(Embedding(input_dim=50, output_dim=8))
model.add(LSTM(10))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

model.fit(padded, labels, epochs=10)

print("Model trained successfully")
```

Question 9: Using spaCy, build a simple NLP pipeline that includes tokenization, lemmatization, and entity recognition. Use the following paragraph as your dataset:

"Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India's atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India."

Write a Python program that processes this text using spaCy, then prints tokens, their lemmas, and any named entities found.

(*Include your Python code and output in the code box below.*)

Answer:

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = """Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in
the development of India's atomic energy program. He was the founding director of the
Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the
Atomic Energy Commission of India."""

doc = nlp(text)

print("Tokens and Lemmas:")
for token in doc:
    print(token.text, "->", token.lemma_)

print("\nNamed Entities:")
for ent in doc.ents:
    print(ent.text, "-", ent.label_)

OUTPUT
Homi Jehangir Bhaba - PERSON
India - GPE
Tata Institute of Fundamental Research - ORG
Atomic Energy Commission of India - ORG
```

Question 10: You are working on a chatbot for a mental health platform. Explain how you would leverage LSTM or GRU networks along with libraries like spaCy or Stanford NLP to understand and respond to user input effectively. Detail your architecture, data preprocessing pipeline, and any ethical considerations.

(*Include your Python code and output in the code box below.*)

Answer:

To build a chatbot for a mental health platform, we can combine deep learning models like LSTM or GRU with NLP libraries such as spaCy or Stanford NLP.

Architecture:

Step 1: User Input
User types a message (for example: "I feel stressed today").

Step 2: Text Preprocessing
Use spaCy to:

- Remove stopwords
- Perform tokenization
- Convert words into lemmas

Step 3: Text Representation
Convert the processed text into numerical form using word embeddings (Word2Vec, GloVe, etc.).

Step 4: Deep Learning Model
Use LSTM or GRU to understand the context of the sentence and classify the user's emotion (happy, sad, anxious, stressed, etc.).

Step 5: Response Generation
Based on the detected emotion, the chatbot generates an appropriate response.

Example:
User: "I feel very lonely"
Chatbot: "I understand how you feel. You are not alone. Would you like to talk about what is bothering you?"

Ethical Considerations:

- User privacy must be protected
- No harmful responses should be generated
- The chatbot should not replace professional doctors
- Sensitive data must be stored securely